DS 2026-27 ACCF A1.2



Unax Arruti, Alex Cenera, Asier Corral, Bilal Fouzir

1. Introduction..... 2
2. Architecture and communication..... 2
3. Functionality..... 3
4. Analysis of distributed file system properties..... 3
5. Limitations compared with a cloud file system..... 4
6. Conclusion..... 4

1. Introduction

This report analyzes the file transfer service developed in the SAR/SZA subject as a first, basic form of resource distribution. The goal is to characterize its architecture and functionality, evaluate it against the properties required of a distributed file system, and identify what is missing for it to evolve into a transparent cloud file service such as Dropbox or Google Drive.

2. Architecture and communication

SAR/SZA follows a client-server architecture over TCP.

Protocol design:

- All commands, responses and parameters are ASCII strings. Only file contents may be arbitrary binary data.
- Commands are always 4 characters (`USER`, `PASS`, `LIST`, `DOWN`, `DOW2`, `UPL0`, `UPL2`, `DELE`, `EXIT`) and responses 2 characters (`OK`, `ER`).
- Parameters are appended directly after the command/response, with ? used as a separator when more than one parameter is needed (e.g. in `UPL0`).
- Messages are terminated with CR+LF (`\r\n`), the same line-ending convention used by HTTP, SMTP, POP and others.

Server implementations:

Two variants exist:

- A process-based server (`fork()`), spawning one child process per accepted connection, with a single shared directory for all users.
- A thread-based server (`threading.Thread`), spawning one thread per connection, with a separate directory per authenticated user.

Both maintain a per-session state machine (`Identification to Authentication to Main to Downloading/Uploading`), meaning the server keeps track of which phase each client is in, this makes it a stateful server in *the terminology of Topic 4*.

3. Functionality

The service supports:

- Authentication (`USER`/`PASS`): three predefined users, `anonymous` (accepts any password, download-only) and `sar`/`sza` (fixed passwords, full access).
- Listing (`LIST`): returns filename and size for every file in the working directory.
- Downloading (`DOWN`/`DOW2`, two-phase): the server first reports the file size, the client confirms, then the raw data is sent.
- Uploading (`UPL0`/`UPL2`, two-phase): the client declares filename and size; the server checks it against a maximum file size (10 MiB) and a minimum free space margin (50 MiB), via `os.statvfs`, before accepting the data.
- Deletion (`DELE`) and session termination (`EXIT`).
- Permission enforcement: upload and delete are forbidden for the anonymous user (error code 7).

4. Analysis of distributed file system properties

Property	Assessment in SAR/SZA
Permanent storage	Provided, files persist on the server's disk.
Namespace / file identification	Flat namespace, just filenames. No real file identifier or directory service.
Concurrent access	Possible, but no synchronization, concurrent writes can overwrite each other.
Access protection	Minimal, 3 fixed users, basic read/write permissions only.
Transparency of identification	Limited. Files are identified by their filenames, but the system does not provide a distributed naming mechanism.
Transparency of location	No, only one server, no migration between nodes.
Scalability	Very limited. The service relies on a single server and a single storage location, with no replication, load distribution or mechanism for adding additional storage servers.
Robustness to client failures	Weak. Unexpected client disconnections can terminate the transfer without recovery or resumption.

High availability /fault tolerance	No, no replication. If the server goes down, the service is unavailable.
Consistency	Not handled. No validation policy, no conflict detection.
Security	Minimal, plaintext passwords, no real access control model.

Sharing semantics and server type: SAR/SZA is a stateful server, since it tracks each client's session phase and current transfer. Its sharing model loosely resembles a very simplified session semantics: a client uploads or downloads an entire file at once (no incremental editing), and whoever uploads last silently overwrites the previous version, with no versioning or conflict detection. Since clients do not cache file contents locally, the system does not require cache-validation mechanisms. However, it also provides no explicit mechanism for detecting or resolving concurrent updates to the same file.

5. Limitations compared with a cloud file system

As we mentioned previously SAR/SZA is a very basic distributed system. It only has the basic functionalities so it has clear limitations compared with an advanced cloud file system such as Dropbox. Here are the main limitations:

- No encryption: The passwords and files travel as plaintext without using any encrypted connections like TLS, used by an advanced cloud file system.
- One single server: There is only one server, so if it goes down the whole service stops working until someone fixes it. An advanced cloud file system keeps several copies of the data on different servers.
- Manual work instead of being automatic: The files have to be uploaded or downloaded manually every time instead of being automatic.
- Problems in disconnection: If a client connection drops during a transfer the server will be waiting forever for it. In an advanced one, if the client does not respond during a specific time the server cancels the connection.
- Information gets lost: If two users upload a file with the same name, the later upload can overwrite the existing file without conflict detection or versioning. An advanced system can detect conflicts and preserve previous versions, preventing data loss.

6. Conclusion

SAR/SZA represents a first, basic form of resource distribution: it separates client and server and provides remote file access through a well-defined protocol. However, it falls far short of the properties required from a genuine distributed file system, it lacks location transparency, scalability, fault tolerance, and any explicit consistency management. In practice, it is closer to an FTP-like file transfer utility with basic authentication than to a distributed file system proper. Evolving it toward a cloud-like service will require, at minimum, introducing location transparency, automatic synchronization, and some form of replication and consistency control.